```
1 #include <stdio.h>
2
3 int main()
4 {
5     //Define the array
6     int a[5] = {10, 20, 30, 40, 50};
7
8     int x = *a;
9
10     ...
11
12     return 0;
13 }
```

Code 6

From Code 6, in the expression shown in Line 8, the name of the array 'a' is decayed into the pointer to the first element of the array by the compiler. As a result, dereferencing the pointer yields the value stored at the first location of an array, which is nothing but 10 in the example shown in Code 6.

Let us consider one more example to show how the 'subscript in the array is always the offset from the pointer'.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     //Define the array
6     int a[5] = {10, 20, 30, 40, 50};
7
8     int x = a[2];
9
10     // a[2] <=> *(a + 2)
11
12     return 0;
13 }
```

Code 7

From Code 7, Line 8, the compiler internally treats the name of the array 'a' as the pointer to the first element of the array; the subscript value 2 is added to the pointer as a offset to get the new address. The equivalent of a[2] is nothing but *(a + 2). The pictorial representation of Code 7 is shown in Table 1. So, here, once again, the name of the array decays into the pointer to the first element of the array and then the subscript will be added.

The compiler automatically scales a subscript to the size of the object pointed at. For instance, if sizeof(int) is 4 bytes long, then the subscript 2 in a[2] is actually 2 * 4 bytes long from the base address of the array. The compiler takes care of scaling, before adding the subscript to the base address of the array. The detailed explanation is given in Table 1.

An array name when passed as an argument to the function, it decays into a pointer to the first element of the arry by the compiler.

Table 1: Pictorial explanation for Code 7

Statement	`int a[5] = {10, 20, 30, 40, 50};` `int x = a[2];`
Pictorial representation: int a[5] = {10, 20, 30, 40, 50}; a[0] a[1] a[2] a[3] a[4] 10 20 30 40 50 100 104 108 112 116 *Assumptions:* 1. The base address of the array(a) is 100 and sizeof(int) = 4 bytes.	
Interpretation of a[2] in pointer notation and how the new address is computed is shown below:	
`a[2] => *(a + 2)`	An array name (a) is decayed into the pointer to the first element of the array. The subscript 2 is treated as the offset from the pointer, which is added to the base address of the array.
`=> *(a + 2 * sizeof(int))`	Adding constant means it is multiplied by sizeof(DataType) Data Type: int
`=> *(100 + 2 * 4)`	Assuming sizeof(int) = 4 bytes
`=> *(108)`	We get the new address (108)
`=> 30`	Dereferencing the new address(108), gives the value, as shown in the picture with the cyan coloured cell.
Pictorial representation: *(a + 2) a[0] a[1] a[2] a[3] a[4] 10 20 30 40 50 100 104 108 112 116	
`x = a[2]`	Finally, the 'x' contains the value 30.

An array name in an expression (apart from the place of declaration) is also decayed by the compiler to a pointer to the first element of the array. A subscript is always equivalent to an offset from a pointer. 2D arrays cannot be collected in pointer-to-pointer when passed as an argument to the functions. END

By: Satyanarayana Sampangi

The author is a member of the embedded software team at Emertxe Information Technology (P) Ltd (*http://www.emertxe. com*). His areas of interest are embedded C programming combined with data structures, microcontrollers and Linux Internals. He can be reached at *satya@emertxe.com, satya.2891@gmail.com*.